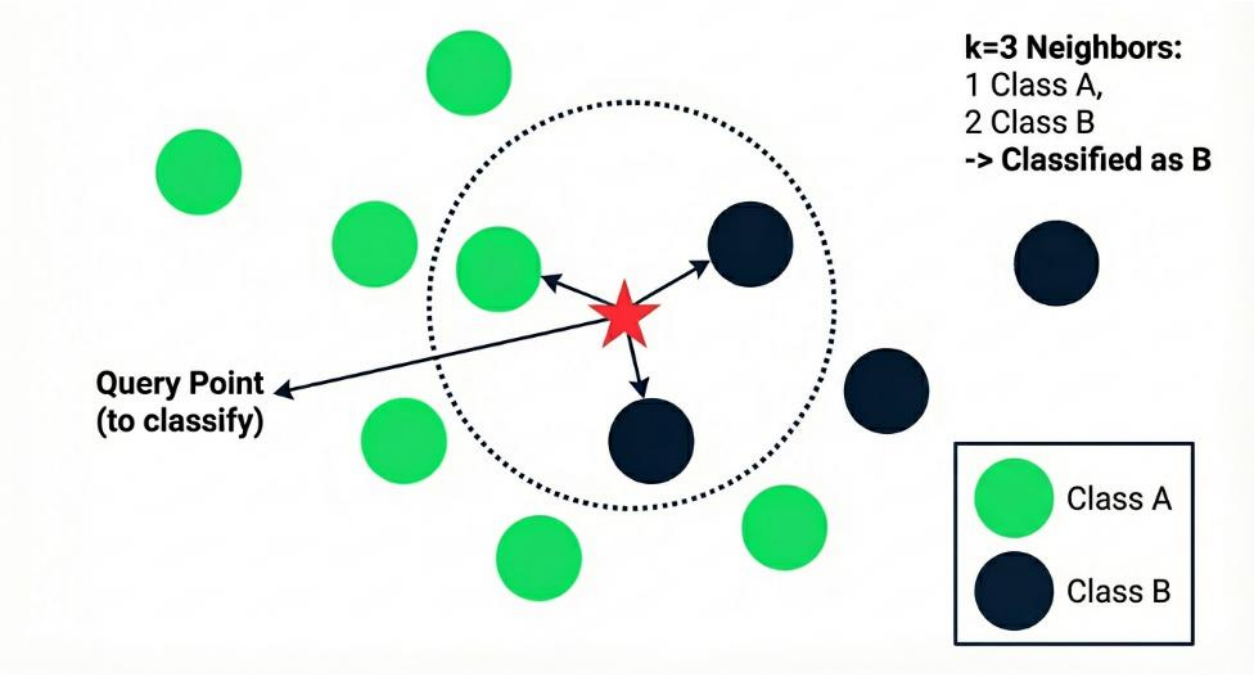
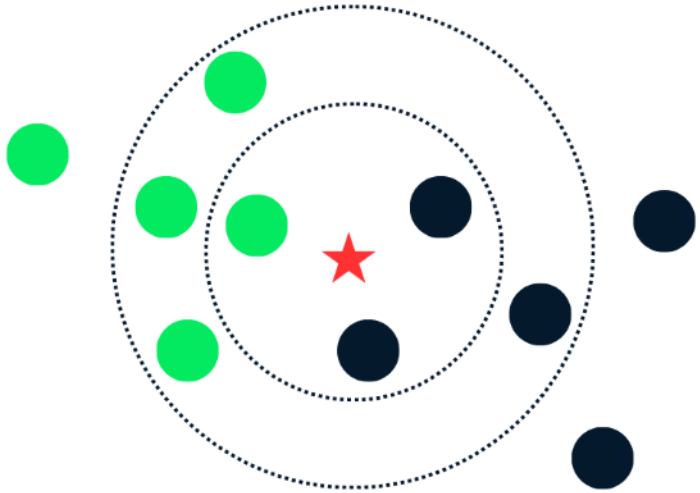
K-Nearest Neighbour (*k*NN) Classifier



Outline

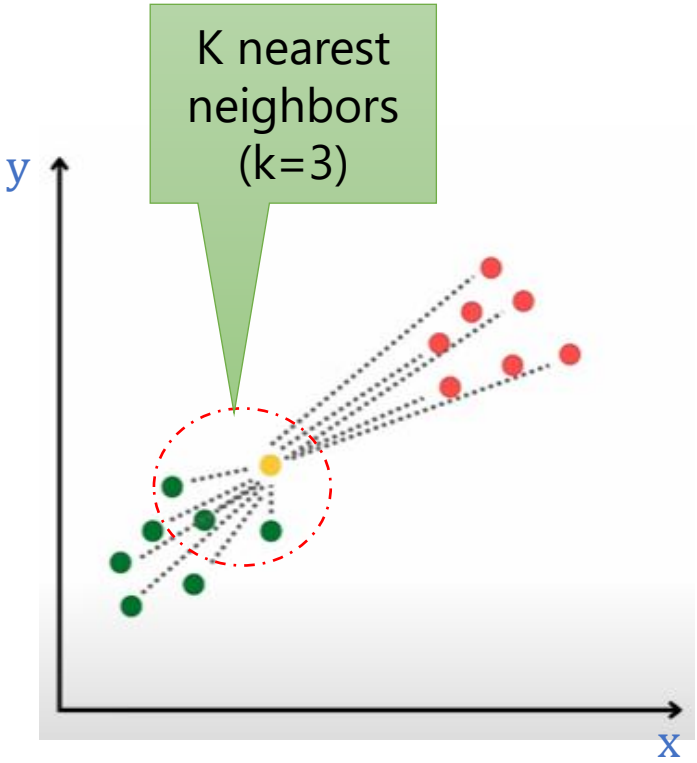
- K-Nearest Neighbour (kNN) Classifier
- Distance Metrics
- Choosing the value of k

K-Nearest Neighbour (*k*NN) Classifier



kNN Algorithm steps

- kNN predicts the label of a new instance based on the labels of the most **similar** training examples.
- Given a data point that we want to classify:
  1. **Compute the distance** between the new instance and all training examples
  2. **Locate the k nearest neighbors** with the smallest distances
  3. For classification: Assign the **majority class** among the k neighbors
  3. For regression: Compute the **average value** of the k neighbors



kNN Key Characteristics (1 of 2)

- 🐌 **Lazy (Instance-Based) Learner**:
  - It does not build an **explicit model** during training
  - All computation happens **at prediction time** using stored training instances

  👉 Ideal when data changes frequently because no retraining is required
- 📊 **Non-parametric** Algorithm:

Makes no assumptions about the underlying data distribution

- 🔍 **Similarity-based** learning:
  - Assumes that similar instances have similar labels
  - Classifies new examples based on their closest neighbors in feature space

kNN Key Characteristics (2 of 2)

- 🧩 **Simple to Implement**
  Easy to understand and apply; often a strong baseline method.

- 📏 **Requires Feature Scaling**
  Distance can be dominated by features with large numeric ranges (e.g., height 1.5–1.8 m, weight 90–300 lb, income 10k–500k QR)

- ⚙️ **Minimal Tuning**
  Only **k** and the **distance metric**

- 👍 **Surprisingly Effective**
  Often performs well—worth trying first on many problems.

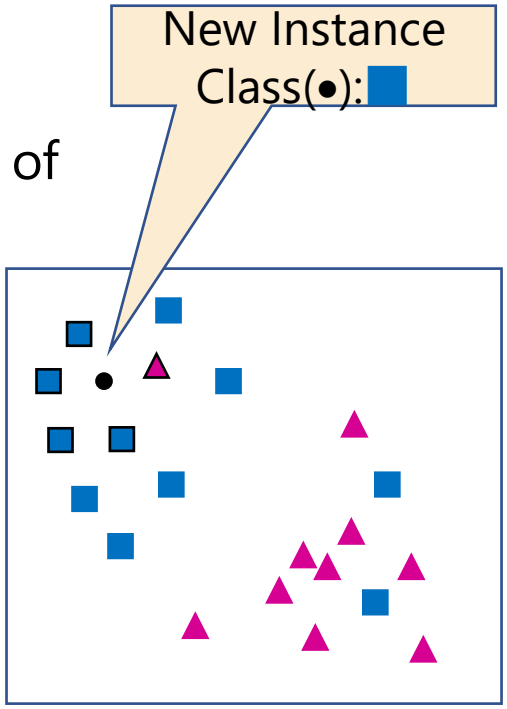
- ⏳ **Expensive at Prediction Time**
  Classifying a new instance requires scanning the **entire dataset**, which becomes 🐢 slow for large datasets.

- 📉 **Performs poorly in high dimensions** since accuracy degrades as the number of features grows.

- 📏 **Sensitive to the distance metric**; different metrics can produce different results

Requirements & Steps

- **Requirements:**
  - The set of training instances
  - Distance Metric to compute distance between instances
  - The value of the hyperparameter *k*, i.e., the number of nearest neighbors to consider
- **To predict an unknown instance:**
  - Compute distances to all training instances
  - Locate ***k*** nearest neighbors
  - **Classify using majority vote** among the k neighbors
  - In the case of a tie:
    - Use odd *k*
    - Choose the class with the **closest neighbor**
    - Pick the class dominant in the entire dataset
    - Decrease *k* by 1 until you break the tie



kNN in Python with scikit-learn

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline

# 1. Load data: 150 flowers, 4 measurements, 3 species
iris = load_iris()
X, y = iris.data, iris.target

# 2. Hold out 20% of the data for testing
# stratify=y ensures that the training and test sets maintain approximately the same
three flower class proportions as the original dataset.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# 3. Scale the features, then fit kNN with k = 5
model = make_pipeline(StandardScaler(),
                      KNeighborsClassifier(n_neighbors=5))
model.fit(X_train, y_train)

# 4. Evaluate on unseen data
print("Accuracy:", model.score(X_test, y_test))  # 0.933
```

Why scale?
kNN relies on distances, so large-scale features would dominate.

fit() = store
A lazy learner just memorizes the training data.

score() = accuracy
Share of test flowers classified correctly.

Choosing k and Predicting a New Flower

```
from sklearn.model_selection import cross_val_score

# Try several k values (5-fold cross-validation)
for k in [1, 3, 5, 7, 9, 11, 15]:
    knn = make_pipeline(StandardScaler(),
        KNeighborsClassifier(n_neighbors=k))
    score = cross_val_score(knn, X_train, y_train,
                            cv=5).mean()
    print(f"k={k:2d}  accuracy={score:.3f}")

# Predict a brand-new flower
# (sepal L/W, petal L/W in cm)
new_flower = [[5.1, 3.5, 1.4, 0.2]]
print(iris.target_names[model.predict(new_flower)[0]])
print(model.predict_proba(new_flower))
```

```
# Output
k= 1  accuracy=0.942
k= 3  accuracy=0.958
k= 5  accuracy=0.967
k= 7  accuracy=0.958
k= 9  accuracy=0.958
k=11  accuracy=0.958
k=15  accuracy=0.958

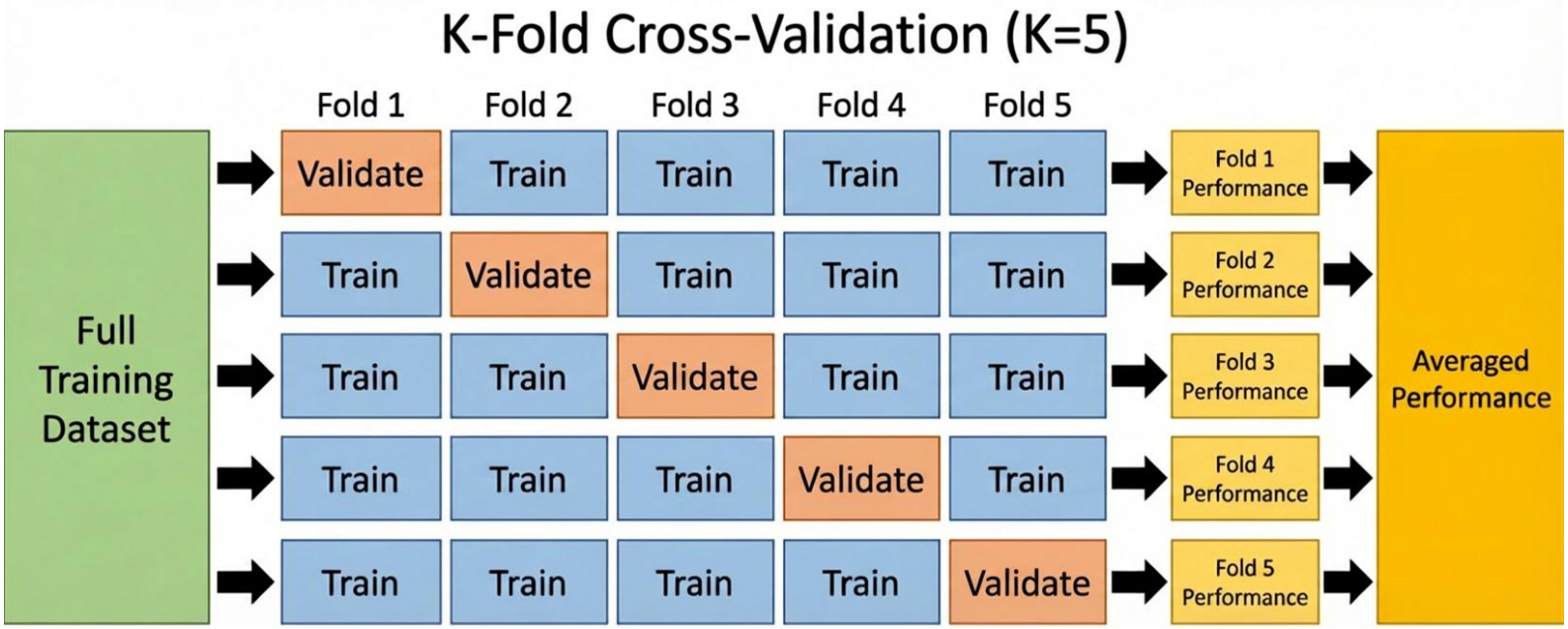
setosa
[[1. 0. 0.]]
```

cross_val_score
Splits the training data into 5 folds: train on 4, test on 1, repeat 5 times. .mean() gives the average accuracy.

Why it is needed
One split can be lucky or unlucky. Averaging gives a reliable score to pick k (k = 5 wins: 0.967) without touching the test set.

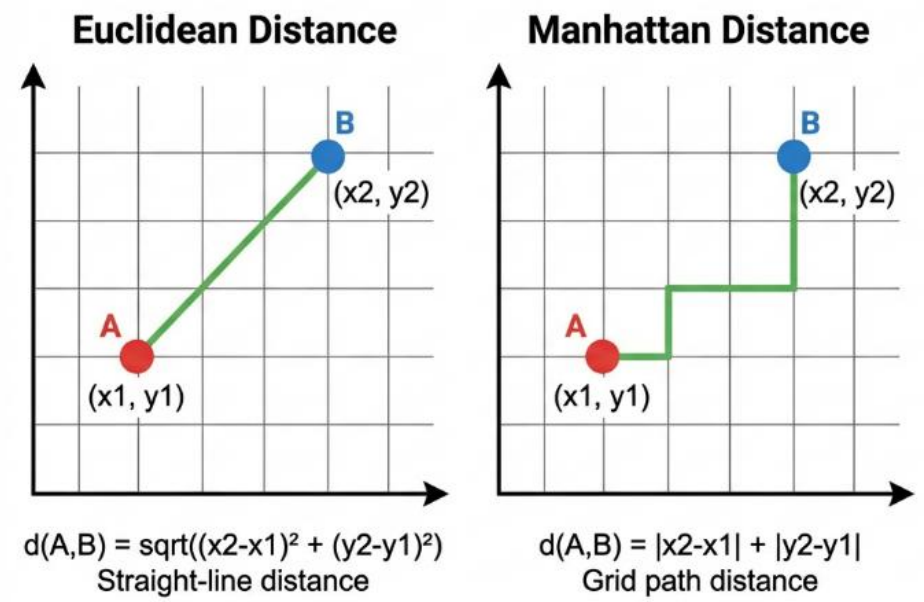
predict_proba
Share of the 5 nearest neighbours voting for each class: here 100% setosa.

K-Fold Cross-Validation



- **What:** split the training data into K equal folds; each fold is the validation set once while the other K-1 train the model.
- **Result:** average the K scores for a more reliable estimate than a single split.
- **Pick the metric:** accuracy misleads on imbalanced data. Precision = TP/(TP+FP) shows how many predicted positives are right; Recall = TP/(TP+FN) shows how many real positives are found; F1 balances both.
- **In practice:** **cross_val_score**(knn, X, y, cv=5, **scoring='f1'**).mean(); use 'precision' or 'recall' as needed.

Distance Metrics



Popular Distance Metrics

Euclidean (L2)

$$d_{Eucl}(a, b) = \sqrt{\sum_{k=1}^{m} (a_k - b_k)^2}$$

- Straight-line distance
- Diagonal moves allowed: the shortest possible path

Example: A = (1, 2), B = (4, 6)

$$d = \sqrt{3^2 + 4^2} = 5$$

Best for: continuous features, low dimensions
Outliers: sensitive, squares large gaps

Manhattan (L1)

$$d_{Manh}(a, b) = \sum_{k=1}^{m} |a_k - b_k|$$

- Sum of absolute differences
- Moves only along axes, like city blocks: no diagonal shortcut

Example: A = (1, 2), B = (4, 6)

d = |4 − 1| + |6 − 2| = 3 + 4 = 7

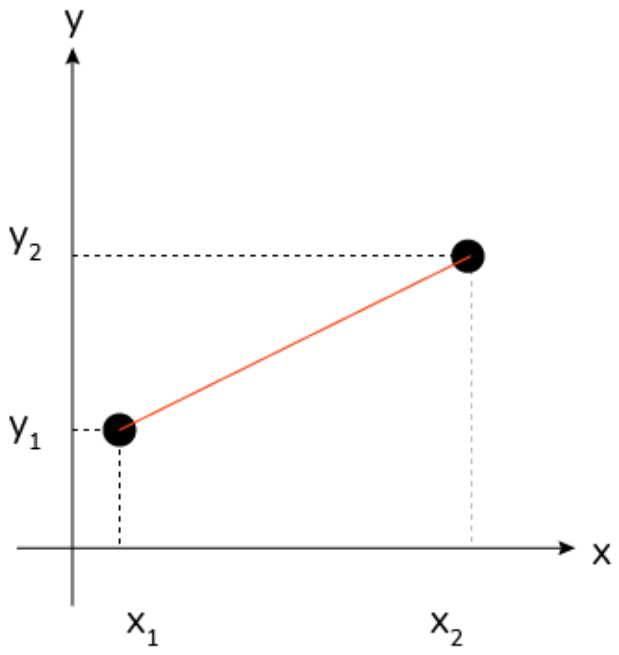
Walk 3 blocks east + 4 blocks north = 7 blocks

Best for: sparse, high-dimensional or noisy data
Outliers: more robust to noise

In kNN, choose it with **metric="euclidean"** or **metric="manhattan"**

Euclidean Distance

- **Euclidean Distance** is a measure of the **straight-line distance** between two points $(x_1, y_1)(x_2, y_2)$ in a two-dimensional space



It's derived from the **Pythagorean** theorem

$$\text{Distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- Example: A = (1, 2), B = (4, 6)
  - $d = \sqrt{(4 - 1)^2 + (6 - 2)^2} = \sqrt{25} = 5$
  - Smaller distance = more similar = nearer neighbour in kNN

Manhattan Distance

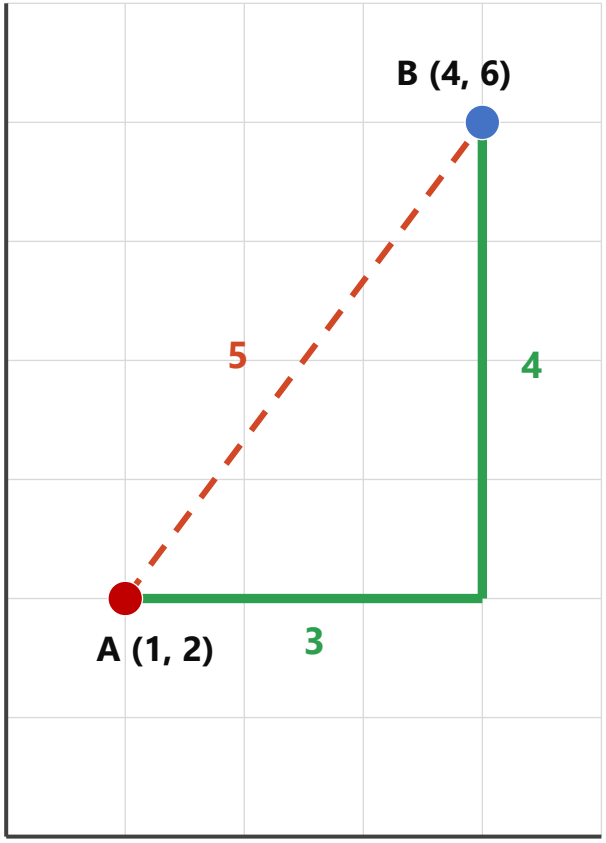
- Sum of the **absolute differences** between coordinates
- Like driving city blocks: horizontal and vertical moves only, no diagonal shortcuts

$$Distance = |x_2 - x_1| + |y_2 - y_1|$$

Example: A = (1, 2), B = (4, 6)

d = |4 − 1| + |6 − 2| = 3 + 4 = 7

Euclidean gave 5 for the same points



Manhattan: 3 + 4 = 7

Euclidean: 5 (dashed)

Use it when features are independent counts, sparse, or noisy: one large gap hurts less than with Euclidean.

Distance for Multiple Features

- When dealing with instances (data objects) with multiple $\mathbf{n}$ features, the Euclidean distance between two instances $\mathbf{p}$ and $\mathbf{q}$ with features ($p_1, p_2, \ldots p_n$) and ($q_1, q_2, \ldots q_n$) respectively, is given by:

$$\text{Euclidean} = \sqrt{\sum_{i=1}^{n} (q_i - p_i)^2}$$

- p_i and q_i represent the values of the ith feature for data objects $\mathbf{p}$ and $\mathbf{q}$ respectively
- $\mathbf{n}$ is the number of features

$$\text{Manhattan} = \sum_{i=1}^{n} |q_i - p_i|$$

Example: p = (2, 5, 1), q = (4, 1, 3)
Euclidean = √24 ≈ 4.90 Manhattan = 2 + 4 + 2 = 8



slides\knn.xlsx

Selecting a Distance Measure

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import cross_val_score

# metric: "euclidean" (p = 2) or "manhattan" (p = 1)
for metric in ["euclidean", "manhattan"]:
    model = make_pipeline(
        StandardScaler(),
        KNeighborsClassifier(n_neighbors=5, metric=metric))
    acc = cross_val_score(model, X, y, cv=5).mean()
    print(f"{metric:10s} accuracy = {acc:.3f}")
```

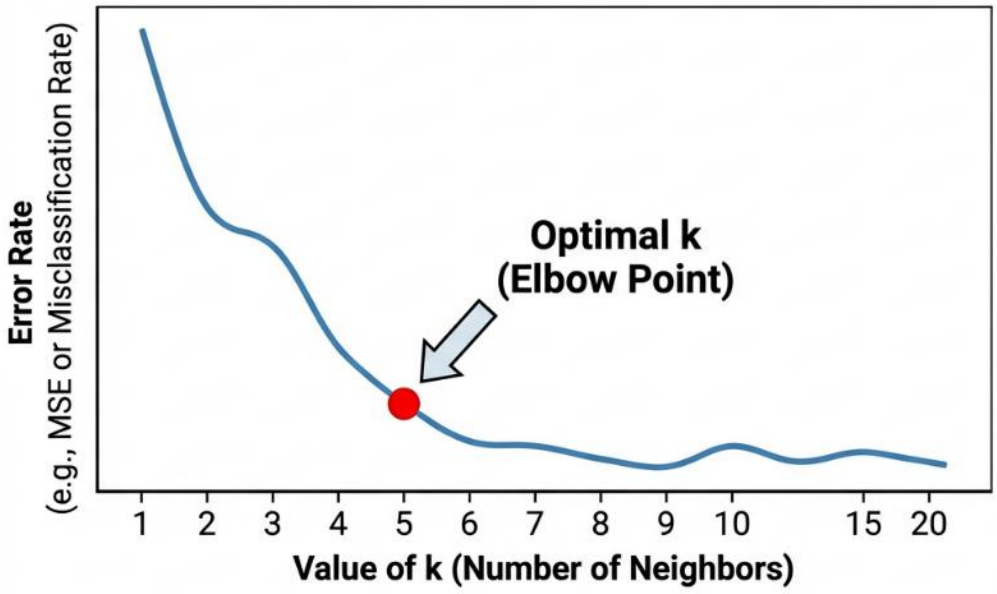
metric="euclidean"
Straight-line distance. scikit-learn's default.

metric="manhattan"
Sum of absolute. Try it for sparse or noisy data.

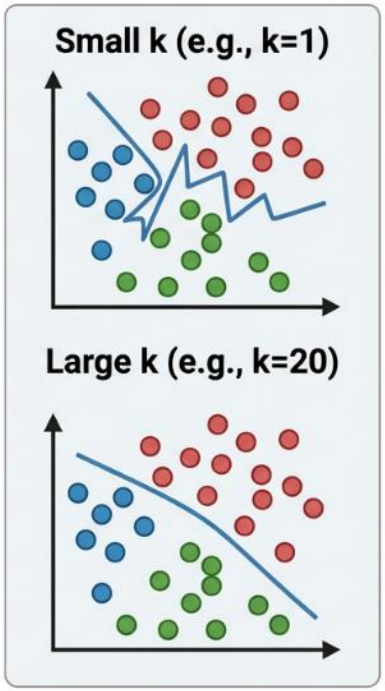
Pick by validation
Scale features first, then keep the metric with the best cross-validated score.

Equivalent: `KNeighborsClassifier(metric="minkowski", p=1)`

Choosing the value of k



Error decreases as k increases, then plateaus. The elbow indicates the best trade-off between bias and variance.



Choosing the value of k

k too small → Overfitting → High Variance

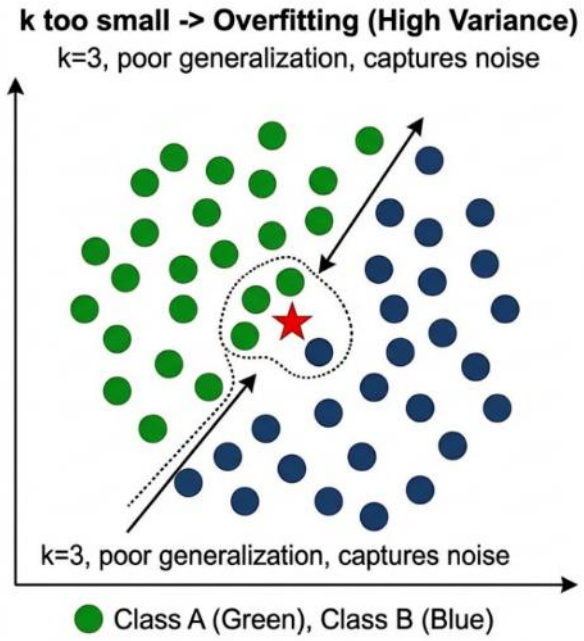
- The model **memorizes** the training data instead of learning general patterns (**poor generalization**)
- **High Variance** = Small data changes ⇒ **big prediction changes**
- Decision boundary become **noisy**

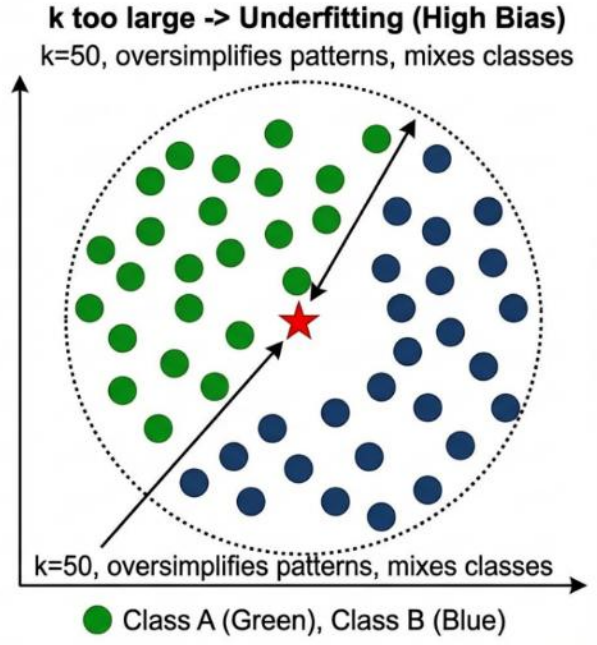
k too large → Underfitting → High Bias

- The model **oversimplifies** because it considers too many neighbors having mixed classes
- **High bias** = Majority vote is influenced by **faraway points**, not just the relevant local ones (distant neighbors dominate the vote)
- Decision boundary **misses important class structure**

🎯 Goal:

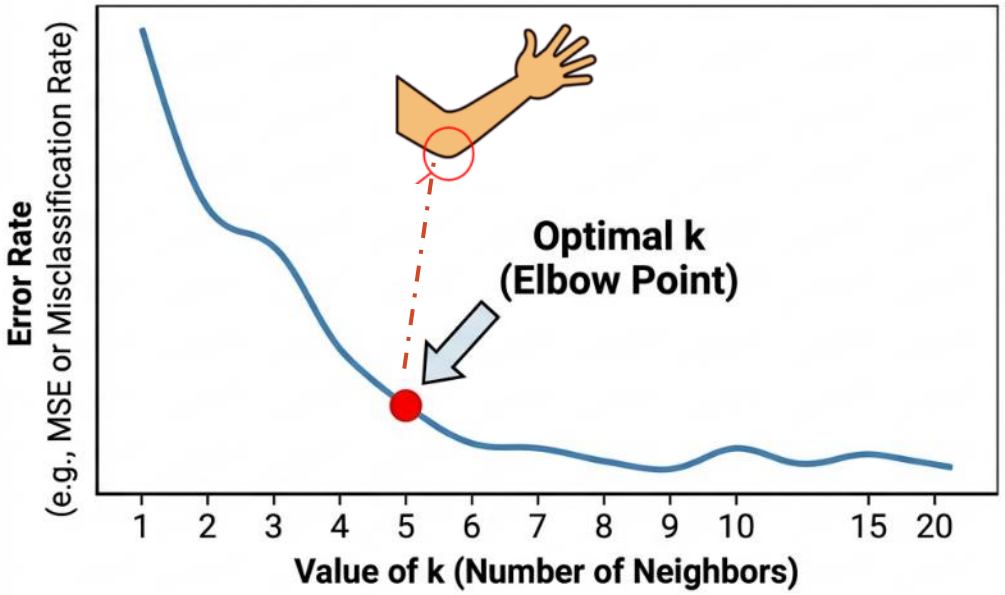
Select a k that balances **bias and variance** for optimal classification performance





The Elbow Method

1. Train kNN with a range of k values
2. Compute the error rate for each k
3. Plot error vs. k



Error decreases as k increases, then plateaus. The elbow indicates the best trade-off between bias and variance.

- Identify the "elbow" point on the curve, where the **error rate stops decreasing sharply** and starts to plateau
- **Choose k at the elbow point** as the optimal number of neighbors
- This elbow indicates the **best trade-off** between overfitting (small k) and underfitting (large k).

Elbow in Practice 1/3: Compute Errors

```
# Error rate = 1 - cross-validated accuracy
ks = [1, 3, 5, 7, 9, 11, 15]
errors = []

for k in ks:
    knn = make_pipeline(
        StandardScaler(),
        KNeighborsClassifier(n_neighbors=k))
    acc = cross_val_score(
        knn, X_train, y_train, cv=5).mean()
    errors.append(1 - acc)

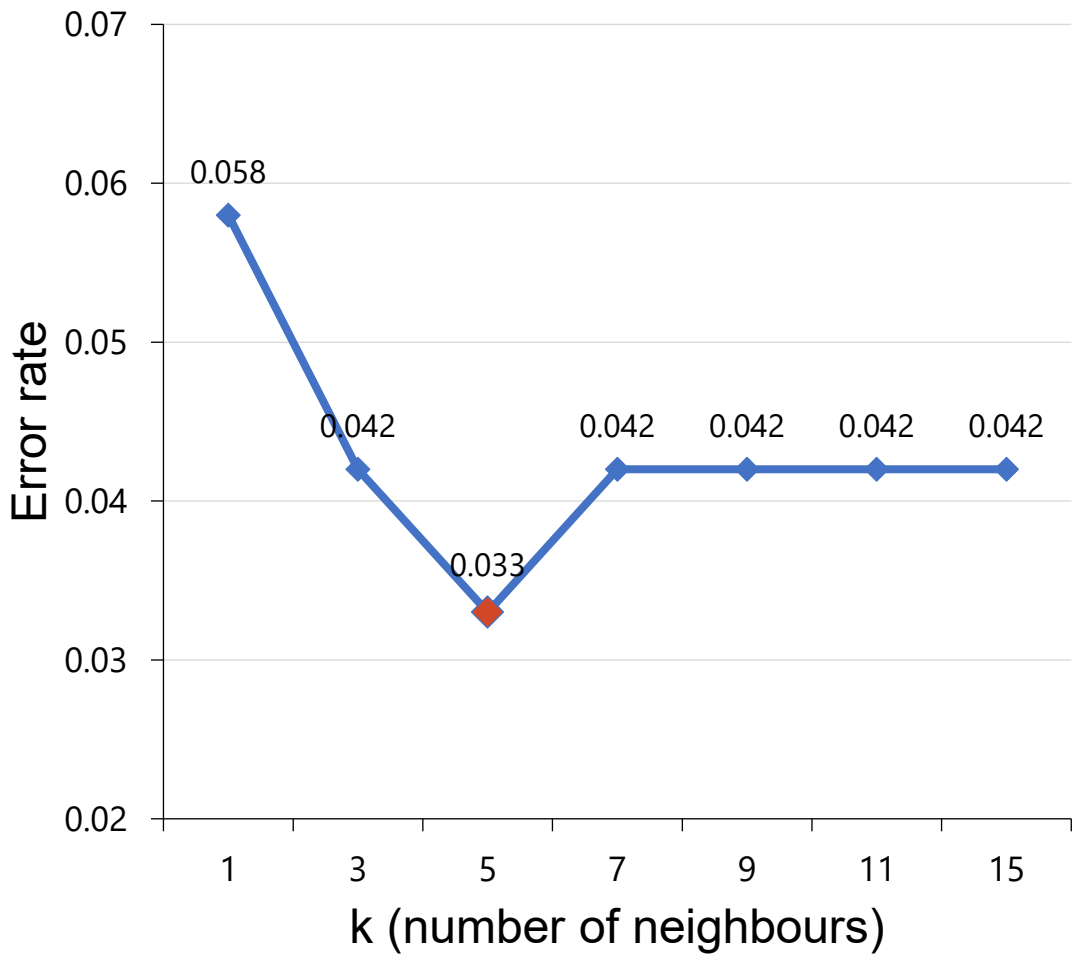
# Plot error vs. k
plt.plot(ks, errors, marker="o")
plt.xlabel("k"); plt.ylabel("Error rate")
plt.show()
```

Simple recipe

1. **Try** a range of k values
2. **Score** each k with cross-validation
3. **Convert** accuracy to error: 1 − accuracy
4. **Plot** error against k

Example: Iris flowers, 5-fold cross-validation on the training set

Elbow in Practice 2/3: Plot the Curve



What the curve shows

k = 1: highest error (0.058), too sensitive to noise

k = 5: lowest error (0.033), the elbow

k > 5: no further gain, curve plateaus

Error rate = 1 − accuracy from slide 9 (e.g. k = 5: 1 − 0.967 = 0.033)

Elbow in Practice 3/3: Pick k

```
# Elbow found at k = 5: train the final
model
model = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=5))
model.fit(X_train, y_train)

# Evaluate once on unseen test data
print(model.score(X_test, y_test))
```

Decision

Elbow: k = 5

Why: lowest error, no gain beyond it

Too small k

Overfits: follows noise, unstable predictions.

Too large k

Underfits: blurs class boundaries.

When flat

If no clear elbow, pick the smaller k

Choose k on training data (cross-validation); touch the test set only once, at the end.